ginyard

Menu About Us @reallygreatsite

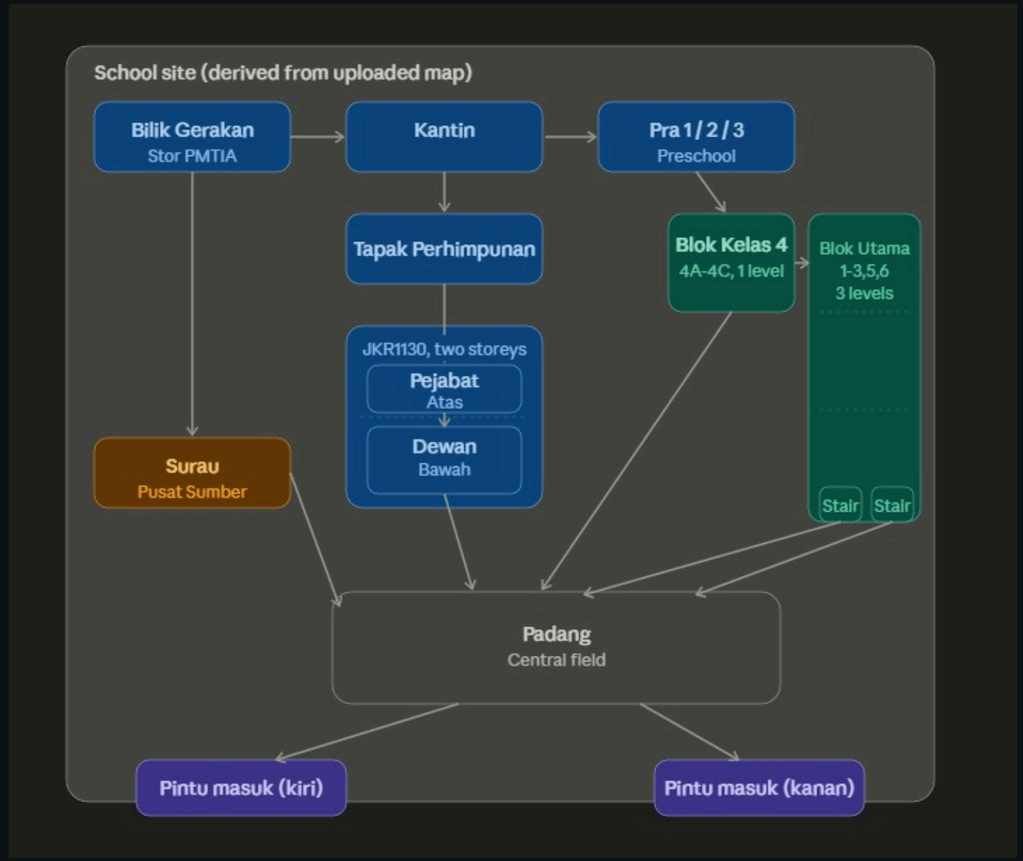
CSC4202 Design and Analysis of Algorithms

SCHOOL EMERGENCY EVACUATION ROUTE PLANNING

MUHAMMAD FITRI BINTI MASHARIL | 224512
MUHAMMAD FARHAN BIN SAZALI | 223293
MUHAMMAD HAZIQ HAKIM BIN YAZID | 222562
NUR SHAHIERA BINTI SHAMADI SHAH | 224328

INTRODUCTION

- Emergency evacuation planning is vital in high-density environments like schools
- The goal is to nsure students/staff reach safety as quickly as possible
- However, human panic leads to choosing familiar but overcrowded routes
- Traditional static exit signs may be ineffective during changing hazards
- The solution is an automated support systems using graph theory



Graph Theory Application

- School building = Geometric Network Graph (GNG)
- Rooms/hallways = Nodes (vertices)
- Connections = Edges
- Dijkstra's Algorithm finds shortest path with modified cost function (distance, fire proximity, blockages

PROBLEM STATEMENT

- Difficulty determining the shortest evacuation path.
- Risk of choosing unsafe routes near the fire source.
- Increased evacuation time during emergencies.
- Congestion and bottlenecks along evacuation routes.
- Higher risk of injuries and exposure to hazardous areas.

OBJECTIVES

- To model the school layout as a weighted graph.
- To apply Dijkstra’s Algorithm to find the shortest evacuation routes.
- To determine the safest path to the assembly area during a fire emergency.
- To improve evacuation efficiency and occupant safety.

DISASTER SCENARIO: CLASS FIRE

Fire originates from Blok Utama (H1).

Possible causes:

- Electrical faults
- Short circuits
- Overheating equipment

Smoke and fire spread to nearby areas.

Blok Utama becomes unsafe and unavailable for evacuation.

Students and staff must evacuate to the Assembly Area (Padang I).

Challenges:

- Unsafe evacuation routes
- Multiple route options
- Panic and confusion
- High number of evacuees

DAMAGE AND IMPACT

Safety Impact

- Burns and physical injuries.
- Smoke inhalation.
- Panic and confusion.
- Delayed evacuation.

Infrastructure Damage

- Damage to classrooms.
- Damage to electrical systems.
- Loss of equipment and documents.
- Structural building damage.

Evacuation Challenges

- Congestion at evacuation routes.
- Bottlenecks at building exits.
- Increased evacuation time.
- Exposure to hazardous areas.
- Difficulty locating safe exits.

WHY THIS NEEDS AN OPTIMAL SOLUTION

Every second counts in an emergency. A manual search for safe routes is error-prone under panic.

Dijkstra’s Algorithm is suitable for this project because it:

- efficiently finds the shortest path in a weighted graph.
- can determine the shortest evacuation route from any location to the assembly area while guaranteeing an optimal solution when all pathway distances are positive.
- provides a reliable and efficient approach for school evacuation planning.

Complexity for this scenario (12 nodes, 15 edges):

- Time: $O(V^2)$ with array-based implementation , negligible for real-time use
- Space: O(V+E) , runs efficiently on standard scholl computers or emergency devices

ALGORITHM COMPARISON

Paradigm	Handle Weights	Handles Blocked Nodes	Suitable For The Scenario
Sorting	No	No	Unsuitable
Divide & Conquer	No	No	Unsuitable
Dynamic Programming (Floyd-Warshall)	Yes	Partially	Too Slow
Graph Traversal (BSF)	No	Partially	Inadequate
Greedy (Djikstra’s)	Yes	Yes	Excellent

DIJKSTRA’S ALGORITHM APPLICATION

V represents the set of vertices (school locations).

E represents the set of edges (walkways).

Edge weights represent walking distances.

Padang (I) acts as the evacuation destination.

H1 (Blok Utama) acts as the fire source and is excluded from route computation.

Pseudocode

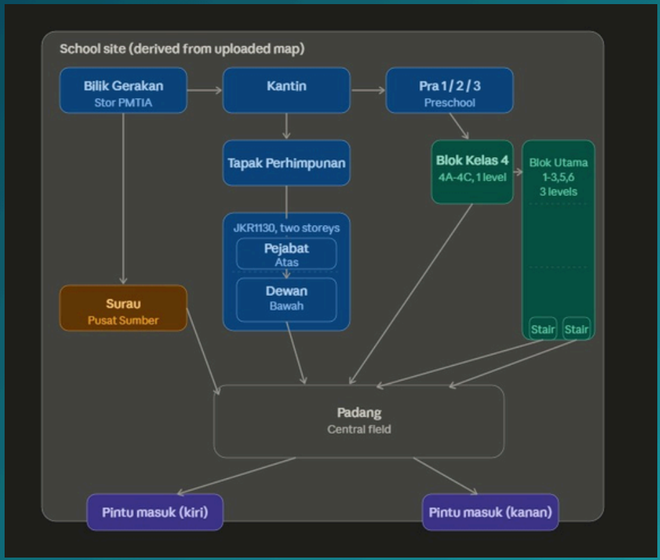
```
DIJKSTRA(Graph, Padang)

Initialize distance = ∞
distance[Padang] = 0

Mark H1 as blocked

Insert Padang into Priority Queue

While Queue not empty

    Select minimum distance vertex

    For each neighbour

        If blocked → Skip

        Calculate New Distance

        If smaller
            Update Distance
            Update Parent

Display Route
Display Distance
```

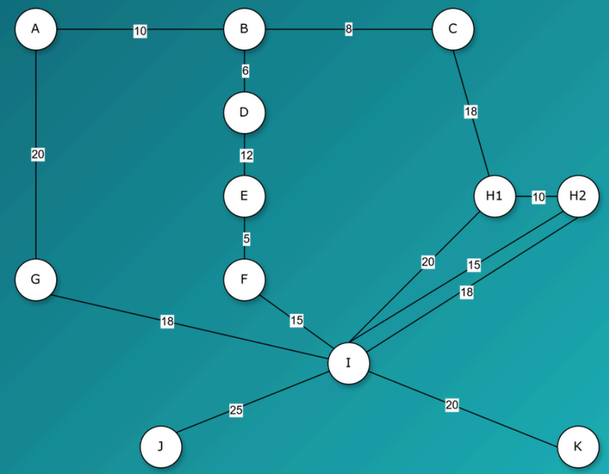
Key Point:

- Initialize distances and parent array.
- Ignore blocked locations (H1).
- Select the vertex with the minimum distance.
- Update shorter distances through edge relaxation.
- Reconstruct and display the evacuation route.

```
================================================
     SCHOOL FIRE EVACUATION PLANNING SYSTEM
       Using Dijkstra's Shortest Path Algorithm
================================================

Fire Source : H1 (Blok Utama)
Safe Area   : I (Padang)
Status      : H1 has been blocked.

================================================
            EVACUATION ROUTE ANALYSIS
================================================

LOCATION : A (Bilik Gerakan)

Shortest Route:
A (Bilik Gerakan)
   |
G (Surau / Pusat Sumber)
   |
I (Padang)

Total Distance : 38 meters
Risk Level     : MEDIUM
Status         : SAFE

------------------------------------------------

LOCATION : B (Kantin)

Shortest Route:
B (Kantin)
   |
D (Tapak Perhimpunan)
   |
E (JKR1130 - Pejabat Atas)
   |
F (Dewan Bawah)
   |
I (Padang)

Total Distance : 38 meters
Risk Level     : MEDIUM
Status         : SAFE
```

Java Implementation





Vertex	Location
A	Bilik Gerakan
B	Kantin
C	Pra 1/2/3
D	Tapak Perhimpunan
E	Pejabat Atas (JKR1130)
F	Dewan Bawah
G	Surau / Pusat Sumber
H1	Blok Utama (Fire Source)
H2	Blok Kelas 4
I	Padang (Safe Area)
J	Pintu Masuk Kiri
K	Pintu Masuk Kanan

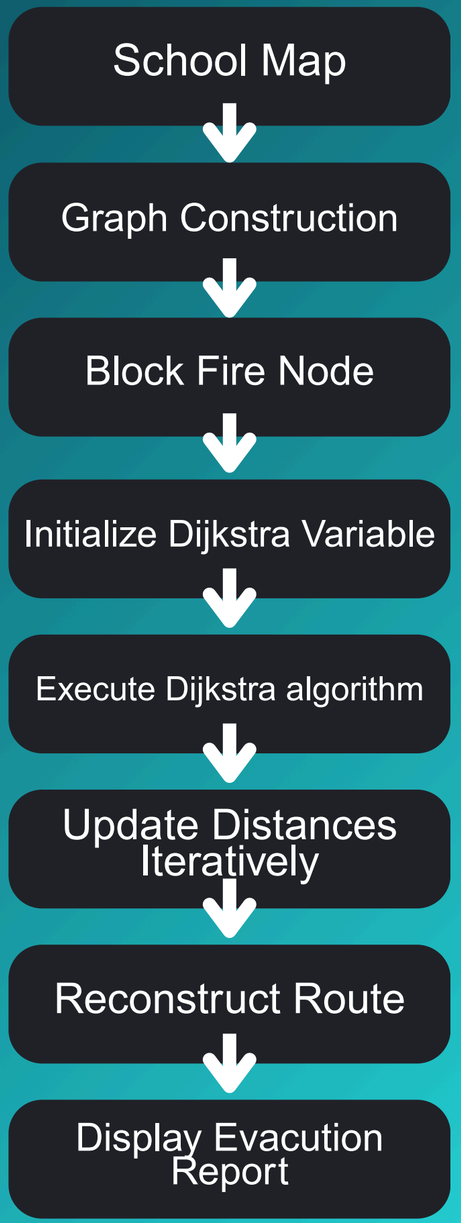
Graph Representation of the School:

- The school evacuation map is represented as a weighted graph
- V (Vertices) represent school locations or buildings.
- E (Edges) represent pathways connecting locations.
- Weights represent the walking distance between locations.

Java Implementation

Table 2.2: Corridor Connections (Edges)

Edge	Distance (m)
A → B	10
B → C	8
B → D	6
A → G	20
D → E	12
E → F	5
F → I	15
G → I	18
C → H1	18
H1 → H2	10
H1 → I	20
H2 → I (Left Stairs)	15
H2 → I (Right Stairs)	18
I → J	25
I → K	20



Important Vertices

H1 (Blok Utama)

- Represents the fire source.
- Marked as a blocked node.
- Excluded during shortest-path computation.
- Ensures routes avoid hazardous areas.

I (Padang)

- Represents the evacuation assembly area.
- Used as the destination vertex.
- Distance initialized to zero.

A, B, C, D, E, F, G, H2, J, K

- Represent locations where students or staff may be located.
- The algorithm computes the shortest route from each location to Padang.

Connection to Java Code

```
addEdge(graph, 0, 1, 10); // A-B
addEdge(graph, 0, 6, 20); // A-G

addEdge(graph, 1, 2, 8);  // B-C
addEdge(graph, 1, 3, 6);  // B-D

addEdge(graph, 3, 4, 12); // D-E
addEdge(graph, 4, 5, 5);  // E-F

addEdge(graph, 5, 9, 15); // F-I
addEdge(graph, 6, 9, 18); // G-I

addEdge(graph, 2, 7, 18); // C-H1
addEdge(graph, 7, 8, 10); // H1-H2

addEdge(graph, 7, 9, 20); // H1-I
addEdge(graph, 8, 9, 15); // H2-I

addEdge(graph, 9, 10, 25); // I-J
addEdge(graph, 9, 11, 20); // I-K
```

- The addEdge() function used to create pathways between 2 point
- These weighted connections form the graph that Dijkstra's Algorithm uses to calculate the shortest evacuation route

Dijikstra’s core

```java
while (!pq.isEmpty()) {

    Node current = pq.poll();
    int u = current.vertex;

    for (Edge edge : graph.get(u)) {

        int v = edge.dest;

        if (blocked[v])
            continue;

        int newDistance =
                distance[u] + edge.weight;

        if (newDistance < distance[v]) {

            distance[v] = newDistance;
            parent[v] = u;

            pq.add(new Node(v, newDistance));
        }
```

Code	Dijkstra Operation
pq.poll()	Select vertex with smallest distance
for(Edge edge...)	Visit neighbouring vertices
newDistance = distance[u] + edge.weight	Calculate alternative route
if(newDistance < distance[v])	Compare distances
distance[v] = newDistance	Update shortest distance
parent[v] = u	Store route information
pq.add(...)	Reinsert updated vertex

Implemented inside the while (!pq.isEmpty()) loop, where the Priority Queue selects the vertex with the smallest distance, and the edge relaxation process compares neighbouring routes and updates the shortest distance whenever a better path is found.

Correctness of Dijkstra’s Algorithm

Location	Theoretical Distance (m)	Theoretical Route	Code Output distance (m)	Code Output Route	Match?
F (Dewan)	15	F → I	15	F → I	/
H2 (Blok Kelas 4)	15	H2 → I	15	H2 → I	/
G (Surau)	18	G → I	18	G → I	/
E (Pejabat)	20	E → F → I	20	E → F → I	/
K (Pintu Kanan)	20	K → I	20	K → I	/
J (Pintu Kiri)	25	J → I	25	J → I	/
D (Tapak Perhimpunan)	32	D → E → F → I	32	D → E → F → I	/
A (Bilik Gerakan)	38	A → G → I	38	A → G → I	/
B (Kantin)	38	B → D → E → F → I	38	B → D → E → F → I	/
C (Pra 1/2/3)	46	C → B → D → E → F → I	46	C → B → D → E → F → I	/
H1 (Blok Utama)	BLOCKED/NO ROUTE	-	BLOCKED (skipped)	-	/

Time Complexity Analysis

Graph Parameters

- V = 12 (Vertices/Locations)
- E = 15 (Edges/Walkways)

Implementation

Min-Heap Priority Queue + Adjacency List

Best Case

O((V + E) log V)

- Occurs when the destination is immediately adjacent to the source.
- Still requires initializing distance values and basic heap operations.
- For this scenario: approximately 97 operations (computationally instantaneous).

Average Case

O((V + E) log V)

- Occurs under normal evacuation conditions.
- Processes all reachable nodes and relaxes most walkway edges using the queue.

Worst Case

O((V + E) log V)

- Occurs when every single area is reachable with zero blocked paths.
- Maintains the same efficiency threshold due to binary min-heap updates.

conclusion

- Dijkstra’s Algorithm is well-suited for school emergency evacuation planning in a weighted, blockage-prone network
- The system successfully computes shortest safe routes from all 11 active locations to the Padang assembly area.
- Theoretical hand-trace and Java program output match perfectly across all nodes.
- The solution is efficient (O((V+E) log V)), scalable, and capable of real-time route updates as conditions change.
- The approach can be extended to larger school campuses by increasing V and E with no change in algorithm logic.

thank you

We know we can’t make everyone happy, but we want everyone to know that we genuinely gave our best effort.